Chapter 5: Retrieval-Augmented Generation (RAG)

Learning Objectives

By the end of this chapter, you will:

- Understand what Retrieval-Augmented Generation (RAG) is and why it is the preferred architecture for enterprise AI solutions.
- Learn how documents are transformed into searchable knowledge using embeddings and vector databases.
- Understand how AI TPMs manage delivery, cost, risks, and stakeholder expectations for RAG programs.

Beginner Explanation

Large Language Models are powerful, but they have one major limitation:

> They only know what they were trained on and cannot automatically access your company's latest information.

For example, if you ask:

> "What is our organization's PTO policy?"

ChatGPT or another LLM will not know your company's internal HR policies.

This is where Retrieval-Augmented Generation (RAG) comes in.

RAG allows an AI system to:

1. Search enterprise documents.
2. Retrieve relevant information.
3. Send the information to the LLM.
4. Generate an accurate response.

Think of RAG as giving an LLM access to a company library.

Simple Analogy

Component	Real-World Analogy
Documents	Books in a library
Chunking	Splitting books into chapters

Component	Real-World Analogy
Embeddings	Library catalog numbers
Vector Database	Library index system
Retriever	Librarian
LLM	Storyteller
Response	Final answer

Why Enterprises Use RAG

- Reduces hallucinations.
- Improves answer accuracy.
- Keeps data current.
- Supports enterprise knowledge.
- Enables secure AI deployments.

Intermediate Explanation

RAG has become the standard architecture for enterprise Generative AI systems.

RAG Workflow

1. Ingest documents.
2. Split documents into chunks.
3. Generate embeddings.
4. Store embeddings.
5. Retrieve relevant content.
6. Pass context to the LLM.
7. Generate the response.

Enterprise Components

Component	Purpose
Blob Storage	Store documents
Azure AI Search	Index and retrieve content
Embeddings Model	Convert text into vectors
Azure OpenAI	Generate responses
Azure Functions	Orchestrate workflows
Monitoring	Track performance

Common Use Cases

Industry	Example
Healthcare	Policy assistants
Banking	Compliance bots
Insurance	Claims support
Retail	Product assistants
IT	Knowledge management
HR	Employee self-service

Enterprise Challenges

- Poor chunking strategies.
- Outdated documents.
- High retrieval latency.
- Security concerns.
- Cost management.
- Search relevance issues.

Why This Matters for an AI TPM

Responsibility	Impact
Delivery	Ensures RAG systems meet accuracy and performance targets.
Risk	Addresses data privacy, hallucinations, and compliance concerns.
Stakeholder Management	Aligns business teams with engineering and AI teams.
Budget	Manages storage, search, embedding, and token costs.

Enterprise Example

Business Problem

A global pharmaceutical company had more than 500,000 internal documents spread across SharePoint, Confluence, and file shares.

Employees struggled to find:

- Standard Operating Procedures
- Compliance documents
- HR policies
- Project documentation

Solution

Build an Enterprise Knowledge Assistant using a RAG architecture.

Architecture

- Azure Blob Storage
- Azure AI Search
- Azure OpenAI
- Azure Functions
- Azure Monitor
- Microsoft Teams

Outcome

Metric	Result
Search Time Reduction	80%
Employee Adoption	91%
Helpdesk Ticket Reduction	48%
Annual Savings	~$1.2M

Lessons Learned

- Chunking strategy significantly impacts quality.
- Search relevance is more important than model size.
- Governance should be established from Day 1.
- Monitoring is essential for production systems.

Azure Implementation

Services Used

Service	Purpose
Azure OpenAI	Response generation
Azure AI Search	Vector and semantic search
Azure Functions	Workflow orchestration
Azure AI Foundry	Prompt flows and evaluations

Implementation Workflow

Step 1: Document Ingestion

Supported sources:

- SharePoint
- Confluence
- PDFs
- Word Documents
- Internal Wikis
- Databases

Step 2: Chunking

Common strategies:

Strategy	Best For
Fixed Size	Small documents
Semantic	Policies and contracts
Sliding Window	Long documents
Hybrid	Mixed workloads

Step 3: Generate Embeddings

Example:

> "Employees are eligible for 20 days of annual leave."

↓

```text id="2s5r2h"
[0.45, 0.67, 0.92, ...]
```

```
This numerical representation allows the system to search for similar
content.

---

#### Step 4: Retrieval

Retrieve the top matching documents before sending them to Azure OpenAI.

---
```

```
#### Step 5: Generate Response

Example:

> Based on the HR policy dated June 2026, employees are eligible for 20 days
of annual leave.

---

## Mermaid Diagram

```mermaid
graph TD

A[Enterprise Documents]

A --> B[Chunking Service]

B --> C[Generate Embeddings]

C --> D[Azure AI Search]

D --> E[Retriever]

E --> F[Azure OpenAI]

F --> G[Generated Response]

G --> H[End User]
```

Advanced Enterprise Architecture

```
graph LR

A[SharePoint]
--> B[Blob Storage]

B --> C[Embedding Model]

C --> D[Azure AI Search]

D --> E[Azure Functions]

E --> F[Azure OpenAI]

F --> G[Teams Application]
```
```

```
G --> H[Employees]
```

TPM Checklist

- [ ] Define business objectives.
- [ ] Identify document sources.
- [ ] Establish chunking standards.
- [ ] Define evaluation metrics.
- [ ] Implement security controls.
- [ ] Estimate monthly costs.
- [ ] Conduct performance testing.
- [ ] Establish governance.
- [ ] Configure monitoring dashboards.
- [ ] Obtain stakeholder approval.

Templates

RAG Readiness Assessment

Question	Status
Are document sources identified?	Yes/No
Is search relevance defined?	Yes/No
Is governance approved?	Yes/No
Are KPIs established?	Yes/No
Is monitoring configured?	Yes/No

Cost Estimation Template

Component	Monthly Cost
Azure OpenAI	
Azure AI Search	
Blob Storage	
Azure Functions	
Monitoring	
Total	

Evaluation Metrics

Metric	Target
Accuracy	>95%
Retrieval Precision	>90%
Latency	<5 Seconds
User Satisfaction	>90%
Availability	99.9%

Common Mistakes

1. Assuming larger models solve retrieval problems.
2. Using poor chunking strategies.
3. Ignoring document freshness.
4. Skipping search evaluations.
5. Underestimating costs.
6. Failing to implement governance.
7. Not monitoring production workloads.

Key Takeaways

- RAG is the preferred architecture for enterprise AI applications.
- Search quality often matters more than model size.
- Effective chunking and embeddings significantly improve results.
- Azure AI Search and Azure OpenAI provide a scalable enterprise solution.
- AI TPMs must manage delivery, governance, cost, and stakeholder expectations.
- Every RAG implementation should include monitoring, evaluation, and security controls.

Footer

---

charan.ai | Page 22